

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Vacaciones Primer Semestre

Documentación Técnica del Proyecto

Invernadero Inteligente IoT

Arquitectura de Computadores y Ensambladores 1

Grupo 19

Repositorio de entrega: github.com/michelin101/ARQUI1V1S_G19

Guatemala, 14 de junio de 2026

Tabla de contenido

Tabla de contenido	2
1. Información general del proyecto	4
2. Resumen del proyecto	4
3. Arquitectura general del sistema	5
4. Estructura del proyecto	7
5. Mapa de pines GPIO – Raspberry Pi 3	8
6. Subsistemas implementados	9
6.1 Monitoreo ambiental (temperatura y humedad)	9
6.2 Monitoreo de humedad de suelo	9
6.3 Sistema de riego.....	10
6.4 Sistema de iluminación.....	10
6.5 Sistema de ventilación.....	10
6.6 Sistema de detección de gas.....	10
6.7 Centro de control físico.....	11
6.8 Estado global del sistema.....	11
7. Comunicación MQTT	12
7.1 Topics de publicación (sensores y estado)	12
7.2 Topics de publicación (actuadores)	12
7.3 Topics de suscripción (comandos).....	12
8. Base de datos – MongoDB Atlas.....	14
9. Dashboard web.....	15
9.1 Panel principal (Dashboard)	15
9.2 Controles	15
9.3 Eventos	16
9.4 Sección de análisis ARM64	16
10. Módulos ARM64	17
10.1 Flujo de integración ARM64.....	17
10.2 Biblioteca compartida utils.s	19
10.3 Tabla de módulos ARM64	19
Módulo 1 — Media ponderada	20
Módulo 2 — Detección de anomalías.....	20
Módulo 3 — Detección de anomalías.....	20
Módulo 4 — Predicción lineal simple.....	20
Módulo 5 — Tendencia acumulada avanzada	20
10.4 Compilación mediante Makefile	20

10.5 Formato del archivo lecturas.csv	21
11. Ejecución de los módulos ARM64 desde el backend	22
12. Instrucciones de instalación y ejecución.....	23
12.1 Clonar el repositorio	23
12.2 Configurar variables de entorno	23
12.3 Instalar dependencias	23
12.4 Compilar los módulos ARM64	23
12.5 Iniciar el sistema de control (Raspberry Pi)	23
12.6 Iniciar el backend Flask.....	23
12.7 Iniciar el dashboard React.....	23

1. Información general del proyecto

Esta sección presenta los datos generales del proyecto y la asignación de responsabilidades de cada integrante del grupo, particularmente en el desarrollo de los módulos de procesamiento estadístico en lenguaje ensamblador ARM64.

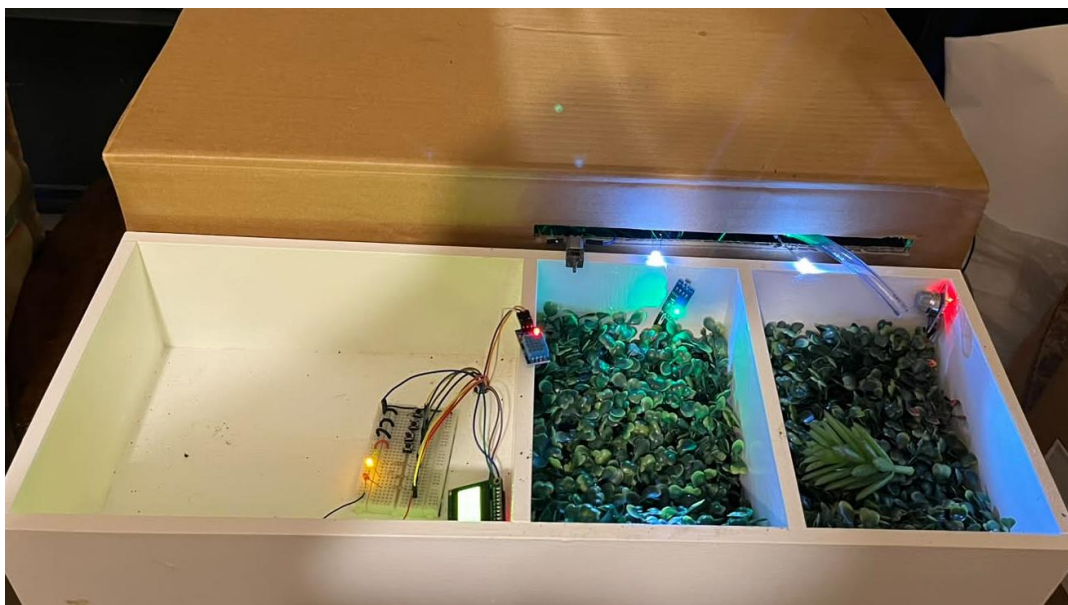
#	Integrante	Carné	Módulo ARM64 a cargo
1	Jennifer Michelle Rosales Juárez	202400063	Módulo 1 – Media ponderada
2	José Bernardo Méndez	202401198	Modulo 2 – Varianza
3	Hector Alexander Pérez Natareno	202406223	Módulo 3 – Detección de anomalías
4	Diego Fernando Curtidor Sagui	202404402	Módulo 4 – Predicción lineal simple
5	Juan Manuel Ordoñez Sandoval	202400006	Módulo 5 – Tendencia acumulada

2. Resumen del proyecto

El proyecto consiste en un invernadero inteligente automatizado, implementado sobre una Raspberry Pi 3 con Python como lógica central. El sistema integra sensores ambientales (temperatura y humedad), sensores de humedad de suelo en dos áreas de cultivo, un sensor de luz (LDR) y un sensor de gas (MQ-135), además de actuadores físicos (bomba de riego, ventilador, LEDs indicadores, buzzer y pantalla LCD con botones de control local).

La comunicación entre la Raspberry Pi y el dashboard web se realiza mediante el protocolo MQTT sobre el broker público broker.emqx.io, bajo el prefijo de topics grupo19/invernadero/. Todas las lecturas, eventos, comandos y registros de actuadores se almacenan en MongoDB Atlas.

Adicionalmente, el sistema incorpora un conjunto de módulos de procesamiento en lenguaje ensamblador ARM64 que realizan análisis estadístico sobre datos reales generados por los sensores. Estos módulos se ejecutan a demanda desde el dashboard, siguiendo el flujo: sensores y MongoDB generan un archivo lecturas.csv de 30 registros, el backend Flask ejecuta los binarios ARM64 sobre ese archivo, los resultados se interpretan, se almacenan nuevamente en MongoDB y finalmente se muestran en la sección de análisis del dashboard.



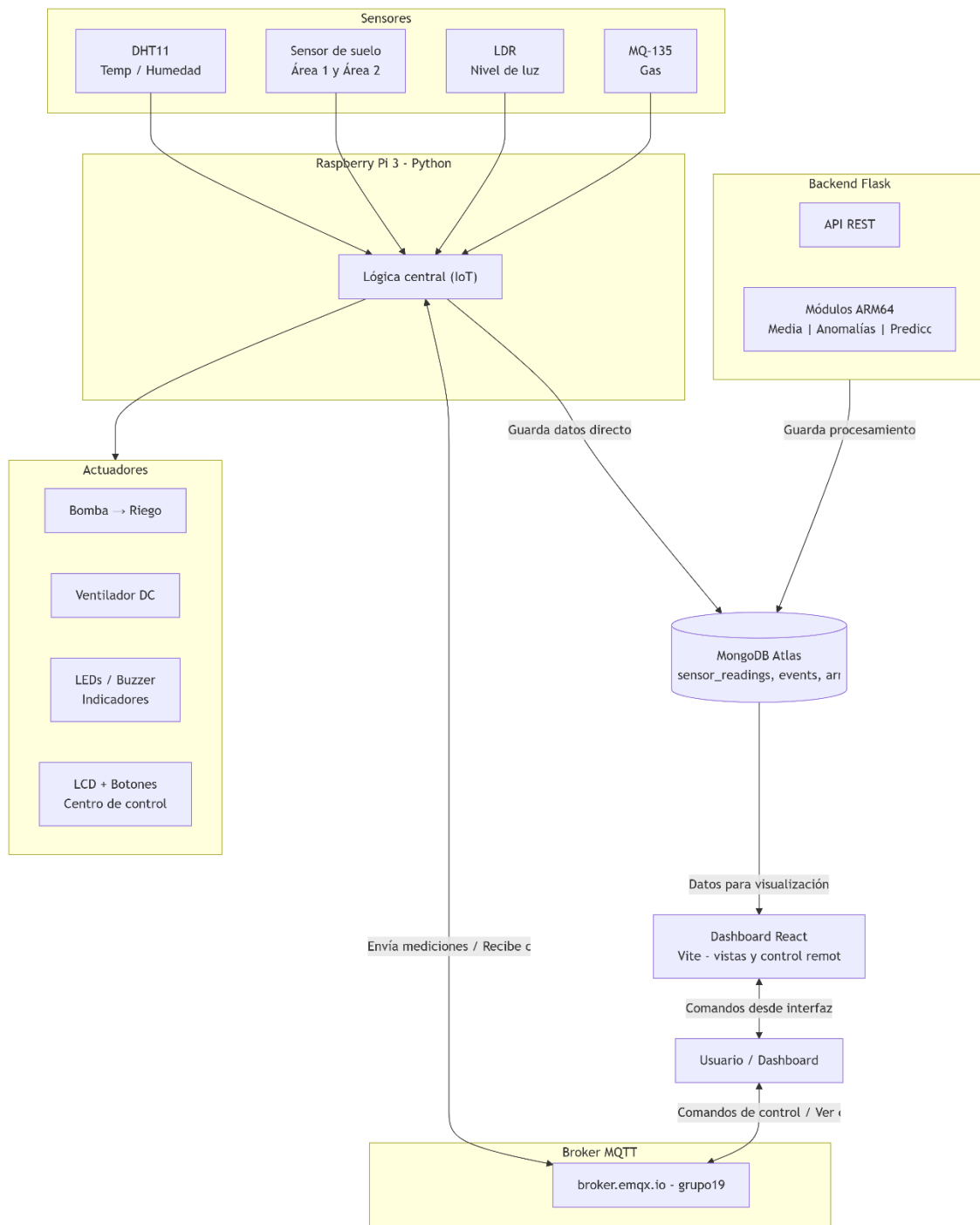
Vista general de la maqueta física del invernadero, mostrando el Área de Cultivo 1, el Área de Cultivo 2 y el Centro de Control.

3. Arquitectura general del sistema

La arquitectura del sistema está organizada en cinco capas funcionales: adquisición de datos (sensores), procesamiento y control local (Raspberry Pi con Python), comunicación y persistencia (MQTT y MongoDB Atlas), procesamiento estadístico nativo (módulos ARM64) y presentación (backend Flask y dashboard React).

El flujo general puede describirse en los siguientes términos:

- Los sensores (DHT11, sensores de humedad de suelo, LDR y MQ-135) entregan lecturas analógicas y digitales a la Raspberry Pi.
- La Raspberry Pi, mediante el programa principal en Python, evalúa el estado global del sistema y controla los actuadores (bomba de riego, ventilador, LEDs, buzzer y pantalla LCD).
- Las lecturas, eventos y registros de actuadores se publican por MQTT y se almacenan en MongoDB Atlas de forma periódica.
- El backend Flask expone una API REST que permite generar el archivo lecturas.csv a partir de los datos almacenados en MongoDB y ejecutar los módulos ARM64 sobre dichos datos.
- El dashboard desarrollado en React consume la información almacenada en MongoDB y los resultados de los módulos ARM64, además de permitir el control remoto de los actuadores.



4. Estructura del proyecto

La estructura final del repositorio se organiza en tres componentes principales: backend (servidor Flask y módulos ARM64), frontend (dashboard en React con Vite) e iot (lógica de control que se ejecuta en la Raspberry Pi). A continuación se presenta la estructura real entregada:

```
ARQUIV1S_G19/
├── backend/
│   ├── app.py                Servidor Flask y endpoint /api/analizar
│   ├── database.py           Conexión a MongoDB Atlas
│   ├── generador_csv.py      Genera lecturas.csv desde MongoDB
│   ├── Makefile              Compila los módulos ARM64
│   └── arm64_data/
│       ├── lecturas.csv
│       ├── resultado_media.txt
│       ├── resultado_anomalias.txt
│       ├── resultado_prediccion.txt
│       └── resultado_tendencia.txt
│   ├── build/                Binarios compilados (.gitignore)
│   │   ├── modulo_1_media
│   │   ├── modulo_3_anomalias
│   │   ├── modulo_4_prediccion
│   │   └── modulo_5_tendencia
│   └── src/
│       ├── utils.s           Biblioteca compartida ARM64
│       ├── modulo_1_media.s   Integrante 1
│       ├── modulo_3_anomalias.s Integrante 3
│       ├── modulo_4_prediccion.s Integrante 4
│       └── modulo_5_tendencia.s Integrante 5
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── public/
│   │   ├── favicon.svg
│   │   └── icons.svg
│   └── src/
│       ├── App.jsx           Enrutamiento y estado global
│       ├── App.css
│       ├── index.css
│       ├── main.jsx
│       ├── assets/
│       ├── components/
│       │   ├── charts/
│       │   │   └── Linechart.jsx
│       │   ├── layout/
│       │   │   ├── NavBar.jsx
│       │   │   └── SideBar.jsx
│       │   └── ui/
│       │       ├── ActuatorButton.jsx
│       │       └── SensorCard.jsx
│       ├── pages/
│       │   ├── Dashboard.jsx
│       │   ├── Controles.jsx
│       │   ├── Analisis.jsx
│       │   └── Eventos.jsx
│       ├── services/
│       │   └── mqttService.js
└── iot/
    ├── main.py               Ciclo principal, MQTT y MongoDB (IoT)
    ├── globals.py            Estado global compartido (Singleton)
    ├── sensors.py            Lectura de sensores y evaluación de estado
    ├── actuadores.py         Control de bomba, ventilador, luces, LEDs y buzzer
    └── display.py            Control de la pantalla LCD
```

5. Mapa de pines GPIO – Raspberry Pi 3

La siguiente tabla detalla la asignación real de pines GPIO utilizados por los módulos de sensores y actuadores, según lo implementado en `iot/sensors.py` y `iot/actuadores.py`.

GPIO	Pin físico	Componente	Módulo Python
GPIO 17	11	DHT11 (temperatura y humedad)	sensors.py
GPIO 5	29	Botón modo AUTOMÁTICO / MANUAL	sensors.py
GPIO 6	31	Botón riego manual	sensors.py
GPIO 13	33	Botón encender/apagar luces	sensors.py
GPIO 19	35	Botón silenciar buzzer	sensors.py
GPIO 2 / 3	3 / 5	I2C SDA/SCL – pantalla LCD	display.py
GPIO 25	22	Bomba de riego (relé)	actuadores.py
GPIO 23	16	Ventilador DC	actuadores.py
GPIO 18	12	LEDs de iluminación de cultivo	actuadores.py
GPIO 9	21	Buzzer	actuadores.py
GPIO 27	13	LED verde (estado NORMAL)	actuadores.py
GPIO 22	15	LED amarillo (ADVERTENCIA / RIEGO)	actuadores.py
GPIO 24	18	LED rojo (EMERGENCIA)	actuadores.py
ADS1115 (I2C)	—	Convertidor analógico-digital: sensores de suelo, LDR y MQ-135	sensors.py

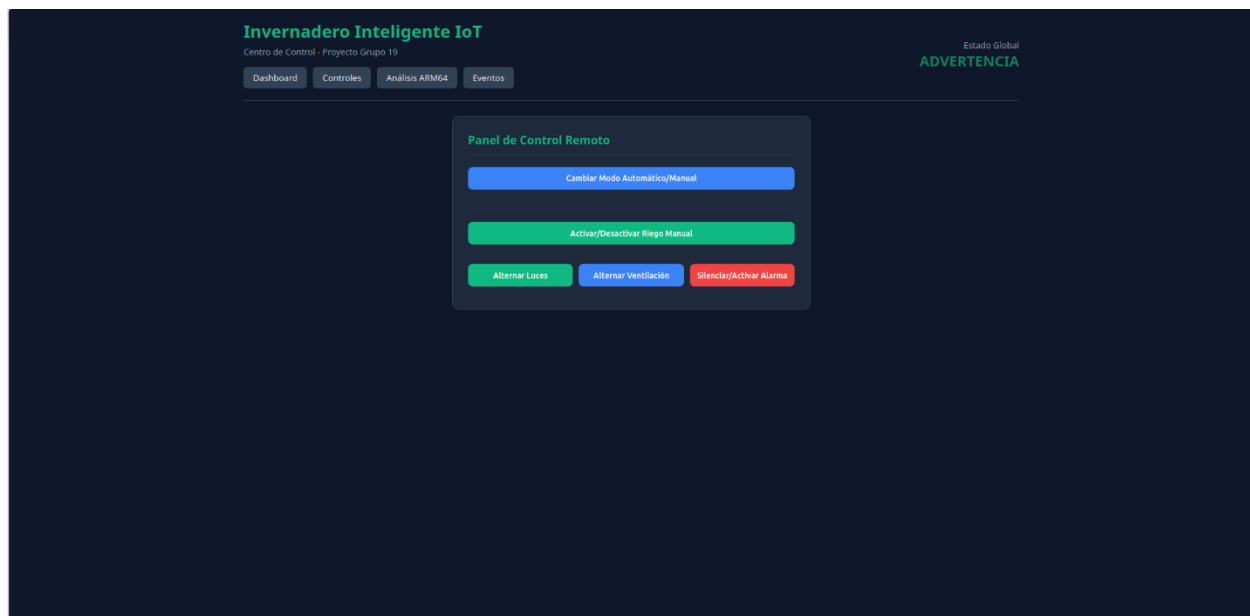
6. Subsistemas implementados

6.1 Monitoreo ambiental (temperatura y humedad)

Componente: sensor DHT11 conectado al pin GPIO 17. Archivo: `iot/sensors.py`.

El sistema lee la temperatura y la humedad relativa del ambiente cada 2 segundos. Estos valores se almacenan en el estado global compartido (`globals.py`), se publican por MQTT, se registran en MongoDB y se muestran de forma rotativa en la pantalla LCD.

Variable	Rango normal	Condición de advertencia
Temperatura	$\leq 30\text{ }^{\circ}\text{C}$	$> 30\text{ }^{\circ}\text{C}$ (activa ventilación)
Humedad ambiental	30 % – 80 %	$< 30\text{ \%}$ o $> 80\text{ \%}$



6.2 Monitoreo de humedad de suelo

Componentes: dos sensores de humedad de suelo conectados al convertidor analógico-digital ADS1115 (canales 0 y 1). Archivo: `iot/sensors.py`.

Cada área de cultivo se monitorea de forma independiente. La clasificación del suelo se determina con los siguientes umbrales:

Estado	Condición y efecto
SECO	Humedad menor al 30 %. En modo automático activa el riego del área correspondiente.
NORMAL	Humedad entre 30 % y 60 %. No se activa el riego.
SATURADO	Humedad mayor al 60 %. Bloquea el riego y genera el estado RIEGO bloqueado por saturación.

6.3 Sistema de riego

Componente: bomba de agua conectada al GPIO 25 mediante relé. Archivo: `iot/actuadores.py`.

La bomba puede activarse de tres formas:

1. Automáticamente, cuando el suelo de alguna de las dos áreas se clasifica como SECO.
2. Manualmente, mediante el botón físico de riego (GPIO 6) cuando el sistema está en modo manual.
3. De forma remota, desde el dashboard, cuando el sistema está en modo manual.

Por seguridad, el riego tiene una duración máxima de 10 segundos por activación y un período de enfriamiento (cooldown) de 15 segundos antes de poder activarse nuevamente.

Estados posibles del riego: `RIEGO_OFF`, `RIEGO_AREA_1`, `RIEGO_AREA_2`, `RIEGO_MANUAL` y `BLOQUEADO_POR_SATURACION`.

6.4 Sistema de iluminación

Componente: LEDs de cultivo conectados al GPIO 18. Archivo: `iot/actuadores.py`.

En modo automático, las luces se encienden cuando el nivel de luz medido por el LDR es menor a 150 lux (estado BAJA) y se apagan cuando el nivel de luz es suficiente. En modo manual, el encendido y apagado se controla mediante el botón físico de luces (GPIO 13) o de forma remota desde el dashboard.

6.5 Sistema de ventilación

Componente: ventilador DC conectado al GPIO 23. Archivo: `iot/actuadores.py`.

En modo automático, el ventilador se activa cuando la temperatura supera los 30 °C. En modo manual, el ventilador puede encenderse o apagarse mediante comandos remotos. En caso de detectarse una emergencia por gas, el ventilador se activa de forma forzada, independientemente del modo de operación, como parte del protocolo de emergencia.

Estados posibles: `VENTILACION_ON`, `VENTILACION_OFF`, `VENTILACION_MANUAL` y `VENTILACION_EMERGENCIA`.

6.6 Sistema de detección de gas

Componente: sensor MQ-135 conectado al ADS1115 (canal 3), buzzer (GPIO 9) y LED rojo (GPIO 24). Archivo: `iot/sensors.py`.

El nivel de gas se mide en partes por millón (ppm) de forma continua. Cuando el nivel supera el umbral de advertencia (800 ppm), el sistema entra en el estado `GAS_ADVERTENCIA`. Cuando supera el umbral de emergencia (1500 ppm), el sistema entra en el estado `GAS_EMERGENCIA`: se activa el buzzer, se enciende el LED rojo, se fuerza la ventilación y el estado global del sistema cambia a `EMERGENCIA`, registrándose el evento en MongoDB.

Estados posibles: `GAS_NORMAL`, `GAS_ADVERTENCIA` y `GAS_EMERGENCIA`.

6.7 Centro de control físico

Componentes: pantalla LCD 16x2 con interfaz I2C (GPIO 2/3), cuatro botones físicos (GPIO 5, 6, 13 y 19), LEDs indicadores (GPIO 22, 24 y 27) y buzzer (GPIO 9). Archivos: `iot/display.py` y `iot/sensors.py`.

La pantalla LCD muestra de forma rotativa diez vistas: temperatura y humedad ambiental, humedad de suelo del Área 1, humedad de suelo del Área 2, nivel de luz, nivel de gas, estado del riego, estado de la ventilación, estado de la iluminación, estado global del sistema y modo de operación. Cuando ocurre un evento de advertencia o emergencia, la pantalla muestra un mensaje fijo de atención durante 3 segundos antes de reanudar la rotación.

Botón	GPIO	Función
Botón 1	GPIO 5	Alternar entre modo AUTOMÁTICO y MANUAL
Botón 2	GPIO 6	Activar o desactivar el riego (modo manual)
Botón 3	GPIO 13	Encender o apagar las luces (modo manual)
Botón 4	GPIO 19	Silenciar la alarma sonora

6.8 Estado global del sistema

El estado global se calcula en cada ciclo en función de la prioridad de las condiciones detectadas, según la siguiente tabla:

Estado	Condición
EMERGENCIA	Nivel de gas en estado GAS_EMERGENCIA (mayor a 1500 ppm).
MODO_MANUAL	El sistema se encuentra en modo de operación manual.
RIEGO_ACTIVO	La bomba de riego está encendida.
ADVERTENCIA	Temperatura mayor a 30 °C, humedad fuera del rango 30 %–80 %, nivel de luz insuficiente, suelo seco o saturado en cualquiera de las dos áreas, o gas en estado GAS_ADVERTENCIA.
NORMAL	Todos los sensores se encuentran dentro de los rangos esperados.

LED	GPIO	Estado que indica
Verde	GPIO 27	NORMAL
Amarillo	GPIO 22	ADVERTENCIA o RIEGO_ACTIVO
Rojo	GPIO 24	EMERGENCIA

7. Comunicación MQTT

El sistema utiliza el broker público broker.emqx.io en el puerto 1883. El cliente utilizado en la Raspberry Pi es paho-mqtt (Python) y en el dashboard es mqtt.js, conectado por WebSocket al puerto 8083. Todos los topics utilizan el prefijo grupo19/vernadero/.

7.1 Topics de publicación (sensores y estado)

Topic	Contenido
grupo19/vernadero/sensores/temperatura	Temperatura medida por el DHT11
grupo19/vernadero/sensores/humedad_ambiente	Humedad relativa medida por el DHT11
grupo19/vernadero/sensores/humedad_suelo_area1	Humedad de suelo del Área de Cultivo 1
grupo19/vernadero/sensores/humedad_suelo_area2	Humedad de suelo del Área de Cultivo 2
grupo19/vernadero/sensores/luz	Nivel de luz medido por el LDR (lux)
grupo19/vernadero/sensores/gas	Estado del sensor de gas (DETECTADO / LIMPIO)
grupo19/vernadero/sensores/gas_ppm	Nivel de gas medido en ppm
grupo19/vernadero/estado/global	Estado global del sistema

7.2 Topics de publicación (actuadores)

Topic	Contenido
grupo19/vernadero/actuadores/riego	Estado actual del riego
grupo19/vernadero/actuadores/riego_area1	Riego activo en el Área 1 (ON/OFF)
grupo19/vernadero/actuadores/riego_area2	Riego activo en el Área 2 (ON/OFF)
grupo19/vernadero/actuadores/ventilador	Estado de la ventilación
grupo19/vernadero/actuadores/luces	Estado de la iluminación (ON/OFF)
grupo19/vernadero/actuadores/alarma	Estado del buzzer (ON/OFF)

7.3 Topics de suscripción (comandos)

Topic	Contenido
grupo19/vernadero/control/remoto	Comandos generales desde el dashboard
grupo19/vernadero/control/manual	Cambio de modo automático/manual (TOGGLE_MODO)

grupo19/vernadero/actuadores/riego/control	Comando de riego (TOGGLE, ON, OFF)
grupo19/vernadero/actuadores/ventilador/control	Comando de ventilación (TOGGLE, ON, OFF)
grupo19/vernadero/actuadores/luces/control	Comando de iluminación (TOGGLE, ON, OFF)
grupo19/vernadero/actuadores/alarma/control	Comando de alarma (TOGGLE, ON, OFF)

grupo19/#

QoS 0

Topic: grupo19/vernadero/sensores/luz QoS: 0

16.2

2026-06-14 15:39:06:404

Topic: grupo19/vernadero/sensores/gas QoS: 0

LIMPIO

2026-06-14 15:39:06:405

Topic: grupo19/vernadero/sensores/gas_ppm QoS: 0

1031.1

2026-06-14 15:39:06:405

Topic: grupo19/vernadero/estado/global QoS: 0

ADVERTENCIA

2026-06-14 15:39:06:406

Topic: grupo19/vernadero/actuadores/riego QoS: 0

RIEGO_OFF

2026-06-14 15:39:06:407

Topic: grupo19/vernadero/actuadores/riego_area1 QoS: 0

OFF

8. Base de datos – MongoDB Atlas

La base de datos se nombra `invernadero_g19` y se conecta mediante la variable de entorno `MONGODB_URI`, definida en el archivo `.env` de cada componente (iot y backend). El acceso se realiza con el driver oficial `pymongo`.

Colección	Contenido
sensor_readings	Lecturas periódicas de temperatura, humedad, suelo, luz y gas, con marca de tiempo
events	Cambios de estado global del sistema (por ejemplo, transición a ADVERTENCIA o EMERGENCIA)
commands	Comandos recibidos desde el dashboard o desde los botones físicos
system_status	Reporte periódico del estado global del sistema
actuator_logs	Historial de activaciones de la bomba, el ventilador, las luces y el buzzer
arm64_results	Resultados del procesamiento estadístico de los módulos ARM64

Estructura representativa de un documento de la colección `sensor_readings`:

```
{
  "fecha_y_hora": "2026-06-13 10:30:00",
  "tipo_de_dato": "lectura_sensores",
  "valor": {
    "temp": 28, "hum": 65,
    "suelo1": 60, "suelo2": 42,
    "luz": 527, "gas": 91
  },
  "origen": "Sensores",
  "estado_relacionado": "NORMAL"
}
```

InvernaderoG19 > Invernadero_g19

View monitoring

Visualize your data

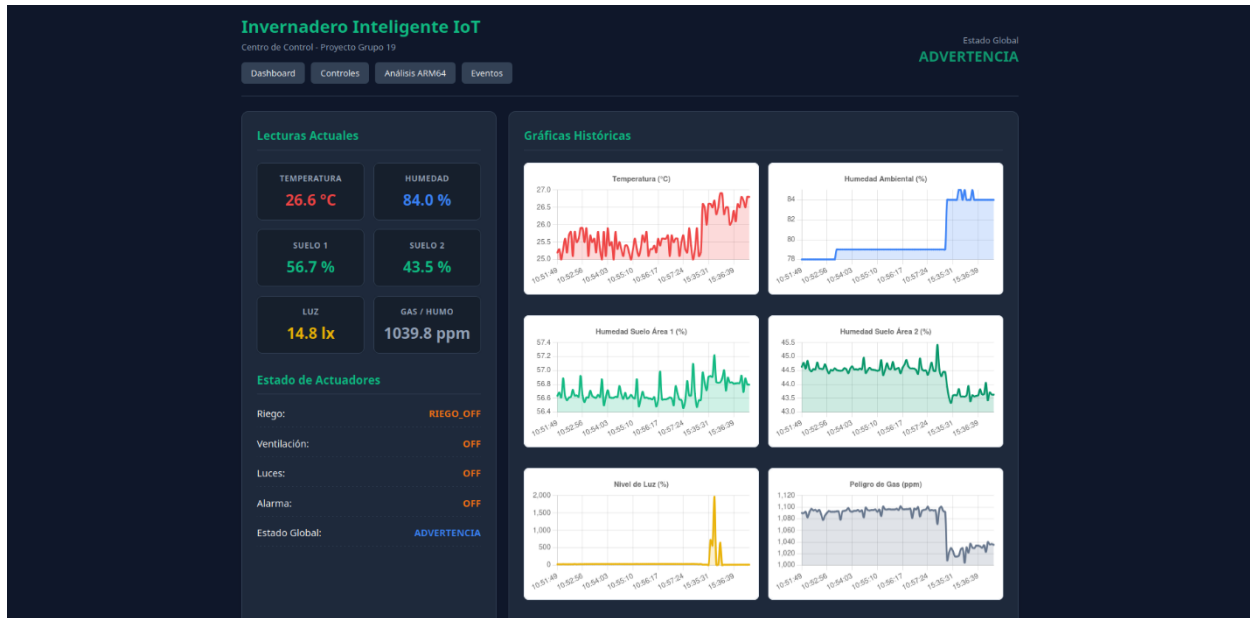
Create collection

Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
actuator_logs	-	36.86 kB	7.21 kB	42	171.00 B	1	36.86 kB
arm64_results	-	36.86 kB	7.69 kB	6	1.28 kB	1	36.86 kB
commands	-	36.86 kB	2.86 kB	14	204.00 B	1	36.86 kB
events	-	36.86 kB	1.91 kB	11	173.00 B	1	36.86 kB
sensor_readings	-	147.46 kB	263.75 kB	1.1K	246.00 B	1	69.63 kB
system_status	-	69.63 kB	184.42 kB	1.1K	172.00 B	1	69.63 kB

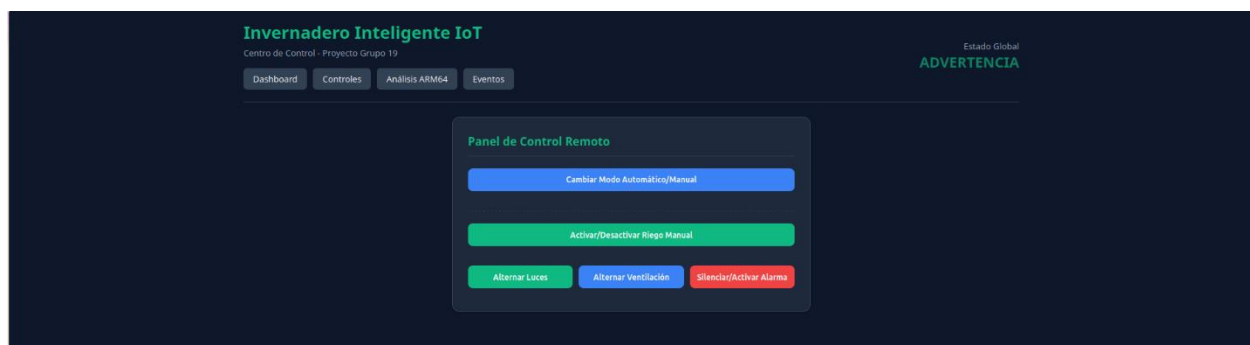
9. Dashboard web

El dashboard fue desarrollado con React y Vite, y se comunica directamente con el broker MQTT mediante mqtt.js para la visualización en tiempo real, y con el backend Flask mediante peticiones REST para el análisis estadístico ARM64. La navegación se organiza en cuatro vistas principales: Dashboard, Controles, Análisis ARM64 y Eventos.



9.1 Panel principal (Dashboard)

Muestra en tiempo real, mediante suscripción a los topics MQTT, el estado global del sistema, la temperatura, la humedad ambiental, la humedad de suelo de cada área, el nivel de luz, el nivel de gas, y el estado de los actuadores (riego, ventilación, luces y alarma).

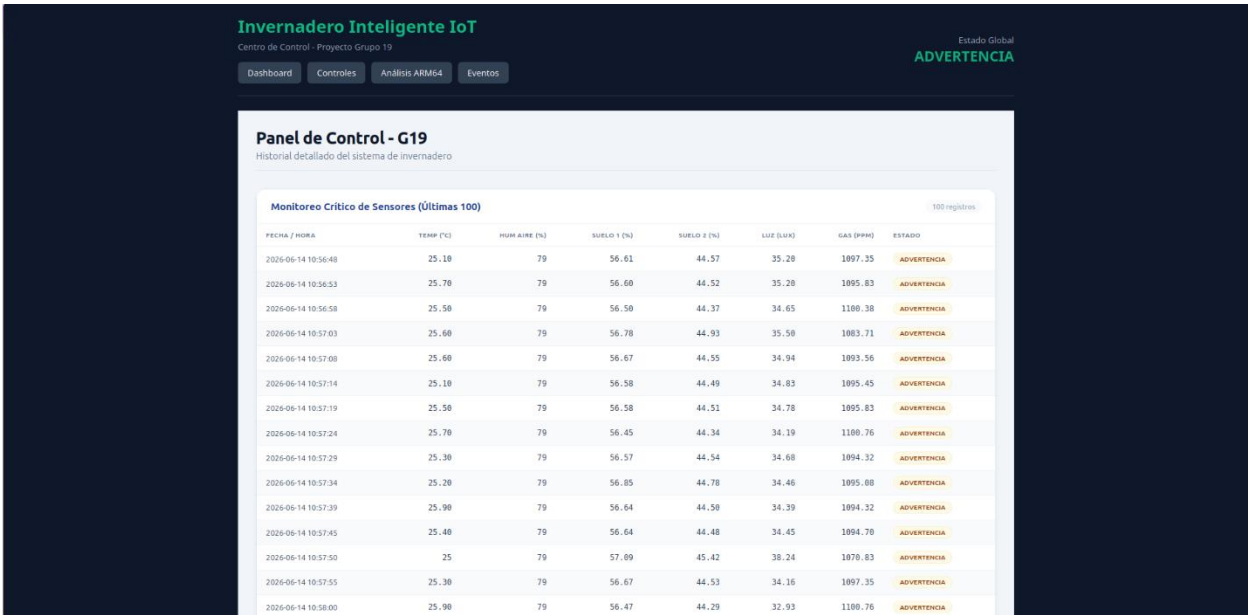


9.2 Controles

Permite, en modo manual, activar o desactivar el riego, encender o apagar las luces, activar o desactivar la ventilación, silenciar la alarma y cambiar entre los modos automático y manual. Los comandos se envían mediante MQTT a los topics de control descritos en la sección 7.

9.3 Eventos

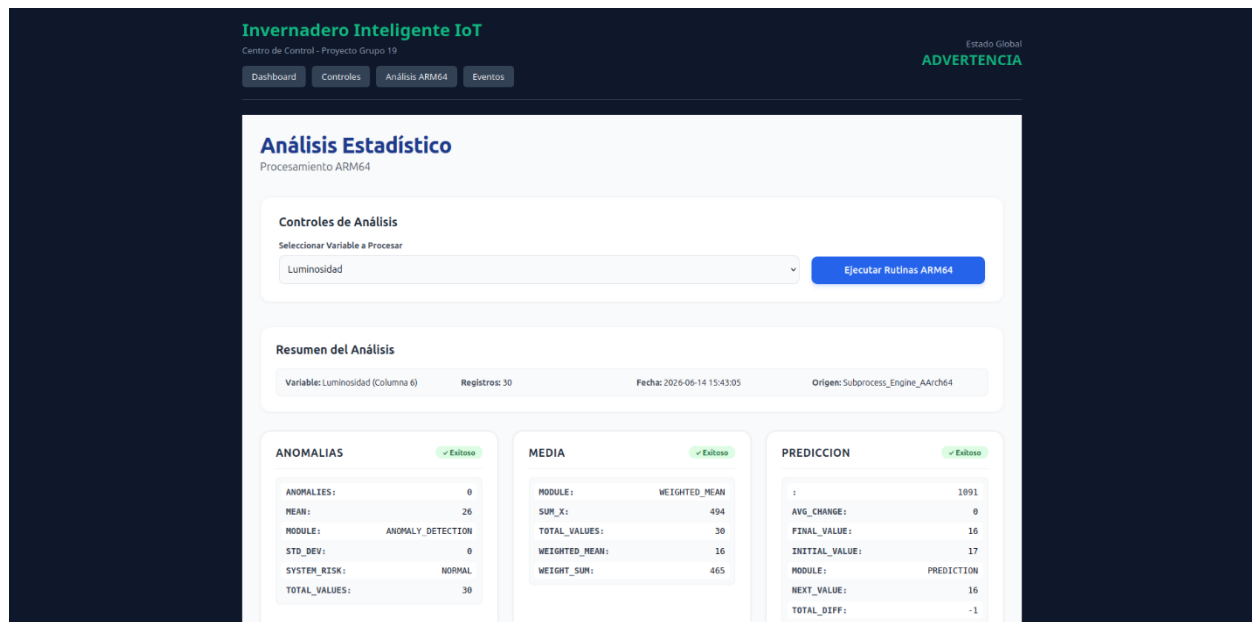
Presenta el historial de eventos, comandos y registros de actuadores almacenados en MongoDB Atlas, consumidos mediante los endpoints /api/historico/sensores, /api/historico/eventos y /api/historico/comandos del backend Flask.



9.4 Sección de análisis ARM64

Permite seleccionar una variable a analizar (temperatura, humedad ambiental, humedad de suelo de cada área, luminosidad o calidad del aire) y ejecutar el análisis estadístico mediante el botón «Ejecutar rutinas ARM64». Al finalizar, muestra el resultado de cada módulo en una tarjeta independiente, indicando si la ejecución fue exitosa o fallida.

Variable	Columna del archivo lecturas.csv
Temperatura	Columna 2 (TEMP)
Humedad de aire	Columna 3 (HUM_AIRE)
Humedad de suelo 1	Columna 4 (HUM_SUELO_1)
Humedad de suelo 2	Columna 5 (HUM_SUELO_2)
Luminosidad	Columna 6 (LUZ)
Calidad del aire (gas)	Columna 7 (GAS)

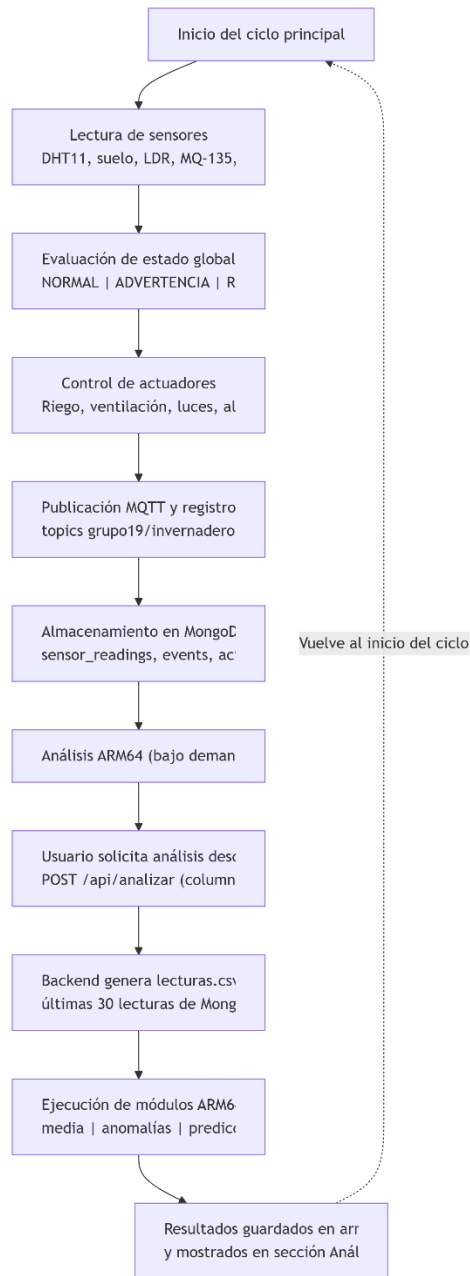


10. Módulos ARM64

El sistema incorpora cuatro módulos de procesamiento estadístico escritos en lenguaje ensamblador ARM64, los cuales operan sobre los datos reales generados por los sensores del invernadero. Todos los módulos comparten una biblioteca común (utils.s) y son ejecutados a demanda por el backend Flask.

10.1 Flujo de integración ARM64

El flujo de integración corregido, acorde a la implementación real del proyecto, es el siguiente:



Descripción del flujo del sistema:

1. La Raspberry Pi ejecuta de forma continua su ciclo principal: lee los sensores, evalúa el estado global y controla los actuadores.
2. Cada lectura se publica por MQTT y se almacena periódicamente en MongoDB Atlas, en la colección sensor_readings.
3. Cuando el usuario solicita un análisis desde la sección ARM64 del dashboard, el backend Flask recibe la petición POST /api/analizar, indicando la columna a analizar.
4. El backend genera el archivo lecturas.csv tomando las 30 lecturas más recientes almacenadas en MongoDB.

5. El backend ejecuta secuencialmente los cuatro binarios ARM64 (modulo_1_media, modulo_3_anomalias, modulo_4_prediccion y modulo_5_tendencia), pasando la columna seleccionada como argumento.
6. Cada módulo lee el archivo lecturas.csv, realiza su procesamiento estadístico y escribe un archivo de resultado en formato CLAVE=VALOR.
7. El backend interpreta cada archivo de resultado, construye un documento consolidado y lo almacena en la colección arm64_results de MongoDB.
8. El dashboard recibe la respuesta de la API y muestra los resultados de los cuatro módulos en la sección de análisis ARM64.

10.2 Biblioteca compartida utils.s

Todos los módulos ARM64 utilizan funciones comunes definidas en la biblioteca utils.s, lo que evita la duplicación de código y garantiza un formato de lectura y escritura consistente entre módulos:

Función	Descripción
utils_read_column_to_stack	Abre el archivo lecturas.csv, recorre las 30 filas y extrae los valores de la columna indicada, almacenándolos en la pila para su posterior procesamiento.
utils_write_result	Crea o sobrescribe el archivo de resultado correspondiente al módulo, escribiendo el contenido del buffer de salida.
utils_itoa	Convierte un valor entero a su representación en texto ASCII para incluirlo en el archivo de resultado.
utils_atoi	Convierte una cadena de texto numérica leída del archivo CSV a su valor entero correspondiente.

10.3 Tabla de módulos ARM64

La siguiente tabla resume los cuatro módulos ARM64 implementados, su responsable y el archivo de resultado que generan. La columna de datos a analizar es seleccionada dinámicamente desde el dashboard (ver sección 9.4) y se pasa como argumento de línea de comandos a cada binario.

Módulo	Descripción	Archivo fuente	Archivo de resultado
1	Media ponderada	modulo_1_media.s	resultado_media.txt
2	Varianza y dispersión estándar	Modulo_2_varianza.s	resultado_varianza.txt
3	Detección de anomalías (Z-score)	modulo_3_anomalias.s	resultado_anomalias.txt
4	Predicción lineal simple	modulo_4_prediccion.s	resultado_prediccion.txt
5	Tendencia acumulada avanzada	modulo_5_tendencia.s	resultado_tendencia.txt

Módulo 1 — Media ponderada

Responsable: Jennifer Michelle Rosales Juárez. Calcula la suma de los valores leídos, la suma de los pesos (fija en 465 para 30 registros) y la media ponderada resultante. Salida: MODULE, TOTAL_VALUES, SUM_X, WEIGHT_SUM y WEIGHTED_MEAN.

Módulo 2 — Detección de anomalías

Responsable: José Bernardo Méndez Juárez. Calcula la media, la varianza y desviación estándar. Salida: MODULE=VARIANCE, TOTAL_VALUES=30, MEAN=31, VARIANCE=18 y STD_DEV=4

Módulo 3 — Detección de anomalías

Responsable: Hector Alexander Pérez Natareno. Calcula la media aritmética y la desviación estándar de los 30 valores leídos, y evalúa cada lectura mediante el método del Z-score: si el valor absoluto de la diferencia entre la lectura y la media, dividido entre la desviación estándar, es mayor o igual a 2, la lectura se considera una anomalía. El módulo también determina un nivel de riesgo del sistema (NORMAL, MEDIUM o HIGH) según la cantidad de anomalías detectadas. Salida: MODULE, TOTAL_VALUES, MEAN, STD_DEV, ANOMALIES y SYSTEM_RISK.

Módulo 4 — Predicción lineal simple

Responsable: Diego Fernando Curtidor Sagui. Calcula la diferencia entre el primer y el último valor de la serie, obtiene el promedio de cambio por lectura y proyecta el siguiente valor esperado de la serie a partir de dicho promedio.

Módulo 5 — Tendencia acumulada avanzada

Responsable: Juan Manuel Ordoñez Sandoval. Recorre la serie de 30 valores y calcula la cantidad de incrementos y decrementos consecutivos, la racha máxima de subidas y bajadas, la diferencia acumulada total entre lecturas consecutivas, y determina la tendencia general de la serie (UP, DOWN o STABLE). Salida: MODULE, TOTAL_VALUES, INCREMENTS, DECREMENTS, MAX_UP_STREAK, MAX_DOWN_STREAK, ACCUM_DIFF y TREND.

10.4 Compilación mediante Makefile

La compilación de los módulos se realiza mediante el Makefile ubicado en backend/, utilizando el ensamblador as y el enlazador ld para la arquitectura ARM64.

```
cd backend

# Compilar todos los módulos de producción
make

# Limpiar binarios y archivos temporales
make clean
```

10.5 Formato del archivo lecturas.csv

El archivo lecturas.csv es generado por el backend (generador_csv.py) a partir de las 30 lecturas más recientes almacenadas en MongoDB, y contiene las siguientes columnas:

```
ID,TEMP,HUM_AIRE,HUM_SUELO_1,HUM_SUELO_2,LUZ,GAS,RIEGO_1,RIEGO_2
1,24,82,58,55,2,1409,0,0
2,24,82,58,55,0,1325,0,0
3,24,82,58,55,9,1376,0,0
...
30,27,74,58,15,347,91,0,0
$
```

Columna	Índice	Descripción	Módulos que la utilizan
ID	1	Número secuencial de la lectura	—
TEMP	2	Temperatura en °C	Selección dinámica desde el dashboard
HUM_AIRE	3	Humedad ambiental (%)	Selección dinámica desde el dashboard
HUM_SUELO_1	4	Humedad de suelo, Área 1 (%)	Selección dinámica desde el dashboard
HUM_SUELO_2	5	Humedad de suelo, Área 2 (%)	Selección dinámica desde el dashboard
LUZ	6	Nivel de luz medido por el LDR	Selección dinámica desde el dashboard
GAS	7	Nivel de gas medido por el MQ-135	Selección dinámica desde el dashboard
RIEGO_1	8	Estado del riego en el Área 1 (0/1)	—
RIEGO_2	9	Estado del riego en el Área 2 (0/1)	—

11. Ejecución de los módulos ARM64 desde el backend

El backend Flask (backend/app.py) expone el endpoint POST /api/analizar, el cual orquesta todo el proceso de análisis estadístico. Al recibir una solicitud con el parámetro columna, el backend ejecuta los siguientes pasos:

1. Valida que el parámetro columna haya sido enviado y corresponda a un valor numérico.
2. Genera el archivo lecturas.csv a partir de las 30 lecturas más recientes de la colección sensor_readings en MongoDB.
3. Ejecuta secuencialmente cada uno de los cuatro binarios ARM64 ubicados en backend/build/, pasando la columna seleccionada como argumento de línea de comandos, con un tiempo límite de 8 segundos por módulo.
4. Lee el archivo de resultado .txt generado por cada módulo y lo convierte a un diccionario de métricas mediante la función extraer_mtricas_txt.
5. Construye un documento consolidado con la fecha, el origen, la columna analizada, el total de registros procesados y los resultados de cada módulo (incluyendo el estado de ejecución, las métricas obtenidas y, en caso de error, el detalle correspondiente).
6. Almacena el documento consolidado en la colección arm64_results de MongoDB y lo retorna como respuesta JSON al dashboard.

Fragmento representativo de la configuración de módulos en backend/app.py:

```
config_rutinas = [
    {"id": "media", "binario": "modulo_1_media", "salida": "resultado_media.txt"},
    {"id": "anomalias", "binario": "modulo_3_anomalias", "salida": "resultado_anomalias.txt"},
    {"id": "prediccion", "binario": "modulo_4_prediccion", "salida": "resultado_prediccion.txt"},
    {"id": "tendencia", "binario": "modulo_5_tendencia", "salida": "resultado_tendencia.txt"}
]

for rutina in config_rutinas:
    ejecucion = subprocess.run(
        [ruta_binario, columna],
        cwd=arm64_data_dir,
        capture_output=True, text=True, timeout=8
    )
    metricas = extraer_metricas_txt(ruta_archivo_salida)
    documento_maestro["resultados"][rutina["id"]] = {
        "estado_ejecucion": "exitoso" if ejecucion.returncode == 0 else "fallido",
        "metricas": metricas,
        "exit_code": ejecucion.returncode
    }
```

Si un binario no se encuentra compilado, si la ejecución excede el tiempo límite o si ocurre un error durante el procesamiento, el módulo correspondiente se marca como fallido y el detalle del error se incluye en la respuesta, permitiendo al dashboard mostrar el estado individual de cada módulo sin afectar la ejecución de los demás.

12. Instrucciones de instalación y ejecución

A continuación se describen los pasos necesarios para configurar y ejecutar el proyecto completo, considerando los tres componentes: el módulo de control IoT (ejecutado en la Raspberry Pi), el backend Flask y el dashboard web.

12.1 Clonar el repositorio

```
git clone https://github.com/michelin101/ARQUIV1S_G19
cd ARQUIV1S_G19
```

12.2 Configurar variables de entorno

Tanto el módulo `iot/` como el módulo `backend/` requieren un archivo `.env` con la cadena de conexión a MongoDB Atlas:

```
MONGODB_URI=<cadena_de_conexion_mongodb_atlas>
MONGODB_DB=invernadero_g19
```

12.3 Instalar dependencias

Dependencias de Python (ejecutar en `iot/` y en `backend/` según corresponda):

```
pip install -r requirements.txt
```

12.4 Compilar los módulos ARM64

```
cd backend
make
cd ..
```

12.5 Iniciar el sistema de control (Raspberry Pi)

```
cd iot
python main.py
```

12.6 Iniciar el backend Flask

```
cd backend
python app.py
```

12.7 Iniciar el dashboard React

```
cd frontend
npm install
npm run dev
```